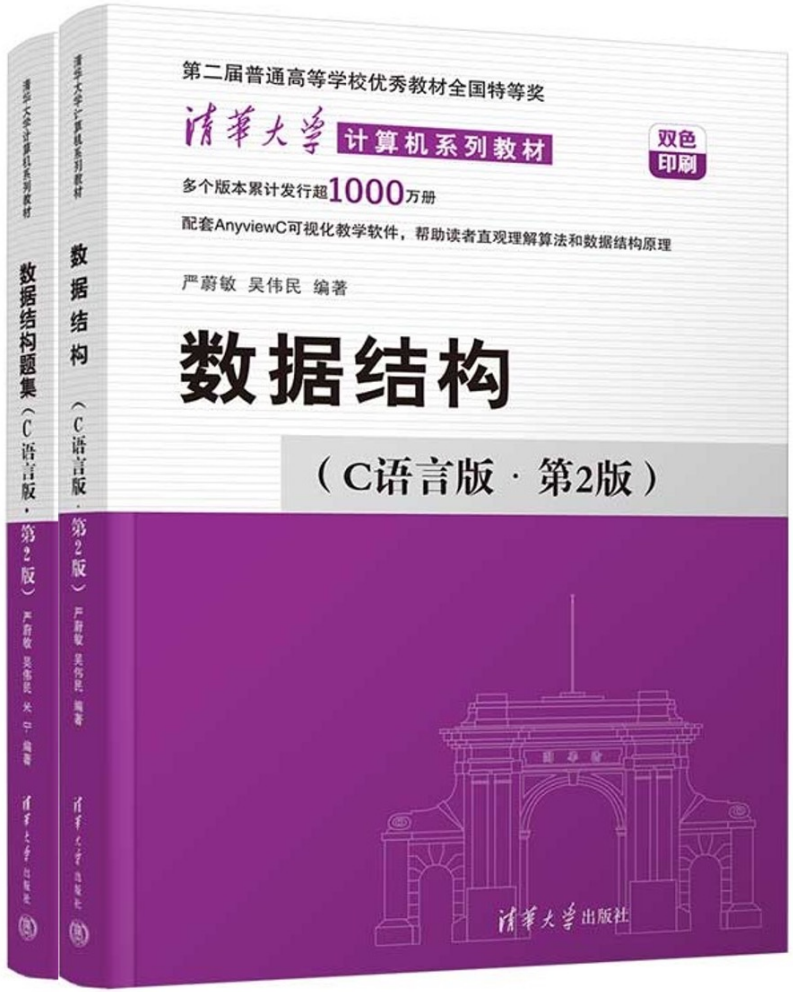


清華大學出版社
TSINGHUA UNIVERSITY PRESS

第2章

线性表

线性表是最典型的线性结构

线性结构的基本特征:

线性结构 是 数据元素的序列

1. 元素集合中必存在唯一的“第一元素”；

2. 元素集合中必存在唯一的“最后元素”；

3. 除最后元素之外，均有唯一的(直接)后继；

4. 除第一元素之外，均有唯一的(直接)前驱。

2.1 线性表的类型定义

清華大學出版社 TSINGHUA UNIVERSITY PRESS

线性表（linear_list），也称为**列表**（list）。
一个线性表是n个数据元素的有限序列。
数据元素的具体含义与应用场景相关，
例如，26个大写英文字母的字母表：

(A, B, C, ···, Z)

整数构成的线性表：

(6, 17, 28, 50, 92, 188)

在稍复杂的线性表中，一个数据元素可以由若干个**数据项**（item）组成。在这种情况下，常把数据元素称为**记录**（record），含有大量记录的线性表又称**文件**（file）。

例如，某校的学生健康情况登记表如图2.1所示，表中每个学生的情况为一个记录，由姓名、学号、性别、年龄、班级和健康状况等6个数据项组成。

姓名	学号	性别	年龄	班级	健康状况
王小林	790631	男	18	计91	健康
陈红	790632	女	20	计91	一般
刘建平	790633	男	21	计91	健康
张立立	790634	男	17	计91	神经衰弱
⋮	⋮	⋮	⋮	⋮	⋮

图2.1　学生健康情况登记表

若将线性表记为

$$(a_1, \cdots, a_{i-1}, a_i, a_{i+1}, \cdots, a_n) \qquad (2\text{-}1)$$

则表中a_{i-1}领先于a_i，a_i又领先于a_{i+1}，称a_{i-1}是a_i的**直接前驱**元素，a_{i+1}是a_i的**直接后继**元素。

当$i = 1, 2, \cdots, n-1$时，a_i有且仅有一个直接后继，

当$i = 2, 3, \cdots, n$时，a_i有且仅有一个直接前驱。

线性表中元素的个数n（$n \geq 0$）定义为线性表的**长度**，$n=0$时称为**空表**。

在非空表中的每个数据元素都有一个确定的位置，如a_1是第一个数据元素，a_n是最后一个数据元素，a_i是第i个数据元素，称i为数据元素a_i在线性表中的**位序**。

抽象数据类型线性表的定义如下：

ADT List {

数据对象：D = { a_i | a_i∈ElemSet, i=1,2,…,n, n≥0 }

数据关系：R1 = { <a_{i-1},a_i> | a_{i-1},a_i∈D, i=2,…,n }

基本操作：

InitList();
操作结果：构造并返回一个空的线性表L。

FreeList(L);
初始条件：线性表L已存在。
操作结果：回收线性表L，返回NULL。

ClearList(L);
初始条件：线性表L已存在。
操作结果：将L重置为空表。

ListEmpty(L);
初始条件：线性表L已存在。
操作结果：若L为空表，则返回TRUE，否则返回FALSE。

ListLen(L);
初始条件：线性表L已存在。
操作结果：返回L中数据元素个数。

GetElem(L, i);
初始条件：线性表L已存在，1≤i≤ListLen(L)。
操作结果：返回L中第i个数据元素的值。若不存在，
则返回出错值errV。

LocateElem(L, e, equal());
初始条件：线性表L已存在，equal()是数据元素判定函数。
操作结果：返回L中第1个与e满足关系equal()的数据元素的
位置。若这样的数据元素不存在，则返回值为0或null。

PrevElem(L, e);
初始条件：线性表L已存在。
操作结果：若e是L的数据元素，且不是第一个，
则用返回它的前驱元素值，否则返回出错值errV。

NextElem(L, e);
初始条件：线性表L已存在。
操作结果：若e是L的数据元素，且不是最后一个，
则用返回它的后继元素值，否则返回出错值errV。

ListInsert(L, i, e);
初始条件：线性表L已存在，1≤i≤ListLen(L)+1。
操作结果：在L中第i个位置插入新的数据元素e，L的长度加1。

ListDelete(L, i);
初始条件：线性表L已存在且非空，1≤i≤ListLen(L)。
操作结果：删除L的第i个数据元素，并返回其值，L的长度减1。
若该元素不存在，则返回出错值errV。

ListTraverse(L, visit());
初始条件：线性表L已存在。
操作结果：依次对L的每个数据元素调用函数visit()。
一旦visit()失败，则操作失败。

}ADT List

清華大學出版社
TSINGHUA UNIVERSITY PRESS

例2-1 假设利用两个线性表LA和LB分别表示集合A和B（即线性表中的数据元素即为集合中的成员），现要求一个新的集合A=A∪B。

这就要求对线性表作如下操作：扩大线性表LA，将存在于线性表LB中而不存在于线性表LA中的数据元素插入到线性表LA中去。只要从线性表LB中依次取得每个数据元素，并依值在线性表LA中进行查访，若不存在，则插入之。

上述操作过程可用算法2.1描述之，其中调用了线性表的4个基本操作。LocateElem的第三个参数是判等函数，判别两个元素是否相等。

本例的集合元素不局限于限定范围的整数，可以是自定义的类型。并且，用了一个已实现的数据结构来表示集合，在后续章节将可看到，可用更多不同的数据结构表示集合。

```
void Union(List La, List Lb) {   // 算法 2.1
  // 将所有在线性表Lb中但不在La中的数据元素插入到La中
  int m = ListLen(La);                    // 求线性表LA长度m
  int n = ListLen(Lb);                    // 求线性表LB长度n
  for (int i=1; i<=n; i++) {              // i依次为Lb中逐个元素的位序
    ElemType e = GetElem(Lb, i);          // 取Lb中第i个数据元素赋给e
  if (0 == LocateElem(La, e, equal))      // 若La中不存在和e相同的数据元素
    ListInsert(La, ++m, e);               // 则m加1后插入e到La位置m（表尾）
  }
}
```

清華大學出版社
TSINGHUA UNIVERSITY PRESS

算法2.1 集合求并集

LA = (1 3 5 7)

算法2.1调用线性表的4个基本操作，实现了求并集。

假设线性表LA和LB分别表示集合A和B，

求并集 A=AUB：

依次取LB的元素，若不在LA中，则插入LA表尾。

LB = (2 3 4 7)

i

```
Status union(List La, List Lb) {   // 算法2.1 求集合的并集
   // 将所有在线性表Lb中但不在La中的数据元素插入到La中
   La_len = ListLength(La);      Lb_len = ListLength(Lb);
   for (i=1; i<=Lb_len; i++) {
      GetElem(Lb, i, e);                 // 取Lb中第i个数据元素赋给e
      if (!LocateElem(La, e,equal)) // La中不存在和e相同的数据元素，则插入
         ListInsert(La, ++La_len, e);
   }
}
```

例2-2　集合“去重”问题。在有的应用中，允许集合含有重复的元素（为与不含重复元素的“纯”集set区别，也称为“包”bag，放宽了数学上对集合元素的唯一性约定。不特别声明的话，两者不一定严格区分）。

所谓集合去重，就是将包“**纯化**”为集。现在的问题是：已知一个“非纯集合”B，试构造一个集合A，使A中只包含B中所有值各不相同的数据元素。

换言之，此问题即为从B中挑选出所有“**彼此相异**”的元素构成一个新的“纯”集合。

如何区分元素的“相异”或“相同”？

一个简单方法即为将每个从B中取出的元素和已经加入到集合A中的元素相比较。

如果还是以线性表LA和LB分别表示集合A和B，那么和例2-1的问题相比，此问题求解的算法应该如何写呢？

容易看出，应对线性表作与上例相同的操作，具体的三步也都相同，**差异仅是：**

例2-1的LA是已有的且可能被添加元素，而此例的LA是要**新建**的。

只需将算法2.1的第一行语句

```
int m = ListLen(La);   // 求线性表LA长度
```

替换为

```
ClearList(La);    int m = 0;   // m置为0，ClearList(La)是确保La是空表
```

并传递一个空线性表给形参La，就得到本例问题的求解算法。

例2-3　判别两个集合是否相等。两个集合相等的充分必要条件是它们具有相同的元素。当以线性表表示集合时，两个相等的线性表的长度应该相等，且表中所有数据元素都能一一对应，但相同的数据元素在各自的线性表中的"位序"不一定相同。"判别两个线性表中的数据元素是否完全相同"的**算法的基本思想**为：首先判别两表的长度是否相等；**在表长相等的前提下，如果对于一个表中的所有元素，都能在另一个表中找到和它相等的元素**的话，便可得到"两个线性表表示的集合相等"的结论（表长相等就不必两个表双向验证）；反之，只要有一个元素在另一个表中不能找到相等元素时，便可得出"不等"的结论。

```
Status IsEqual(List La, List Lb) {   // 算法 2.2  判别两个线性表是否相等
  // 若线性表La和Lb不仅长度相等，且所含数据元素也相同，则返回 TRUE，否则返回 FALSE
  int m = ListLen(La), n = ListLen(Lb);
  if (m != n)  return FALSE;
  else {
    int i = 1;    Status found = TRUE;
    while (i<=m && found) {
      ElemType e = GetElem(La, i);
      if (LocateElem(Lb, e, equal)) i++;
      else  found = FALSE;
    }
    return found;                  // found表达验
  }
}
```

两个算法的时间复杂度取决于抽象数据类型List定义中基本操作的实现代码的执行时间。

假如LA和LB的长度分别为m和n，GetElem和ListInsert这两个操作的执行时间与表长无关，LocateElem的执行时间和表长成正比，则两个算法的时间复杂度都是$O(m \times n)$。

后面将会看到，改变存储结构或元素排列次序，相应算法的效率可得到数量级意义的提升。

2.2 线性表的顺序表示和实现

线性表的顺序表示指的是用一组地址连续的存储单元依次存储线性表的数据元素。

假设线性表的每个元素需占用l个存储单元，并以所占的第一个单元的存储地址作为数据元素的存储位置，则线性表中第i+1个数据元素的存储位置$LOC(a_{i+1})$和第i个数据元素的存储位置LOC(a_i)之间满足下列关系：

$$LOC(a_{i+1}) = LOC(a_i) + l$$

一般来说，线性表的第i个数据元素a_i的存储位置为

$$LOC(a_i) = LOC(a_1) + (i-1) \times l \qquad (2\text{-}2)$$

式中$LOC(a_1)$是线性表的第一个数据元素a_1的存储位置，通常称作线性表的**起始位置**或**基地址、基址**。

线性表的这种机内表示称作线性表的顺序存储结构或顺序映像（sequential mapping），通常，称这种存储结构的线性表为**顺序表**。它的特点是，为表中相邻的元素a_i和a_{i+1}赋以相邻的存储位置$LOC(a_i)$和$LOC(a_{i+1})$。

也就是说，以元素在计算机内“**物理位置相邻**”来表示线性表中数据元素之间的**逻辑关系**。

每一个数据元素的存储位置都和线性表的**起始位置**相差一个和数据元素在线性表中的位序成正比的**常数l**（称为**元素大小**，即存储所占字节数，若元素类型为ElemType，可用C语言求类型大小的运算求得：**sizeof**(ElemType)），如图2.2(a)所示。

只要确定了元素的存储起始位置，表中任一数据元素都可随机存取，所以线性表的顺序存储结构是一种**随机存取**的存储结构。

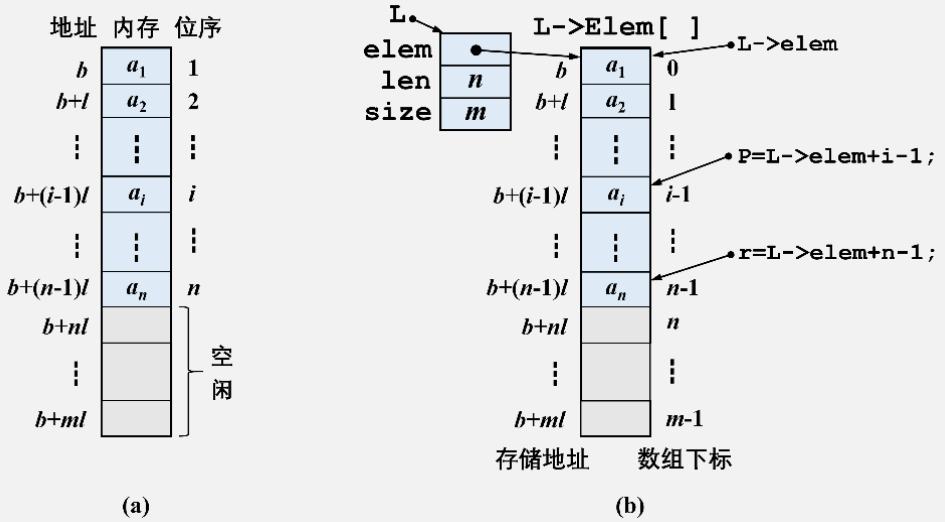


图2.2 线性表的顺序存储结构示意图

(a) 数据元素在顺序存储结构的地址和位序；

(b) 顺序表的结构记录和元素数组存储区及下标和指针示意

```
//----- 线性表的动态分配顺序存储结构 -----
typedef struct {
  ElemType *elem;      // 存储空间基址
  int       len;       // 当前长度
  int       size;      // 当前分配的存储容量，以sizeof(ElemType)为单位
  int       inc;       // 每次存储空间扩容的增量
} *SqList, *List;      // 顺序表类型(定义为指针类型，可实现完全动态存储分配和回收)
SqList  L;             // L被声明为顺序表指针变量，须调用初始化操作才能使用
```

在上述定义中，定义了一个包含存储顺序表结构信息的**结构体**，称为顺序表的**结构记录**，简称**记录**。

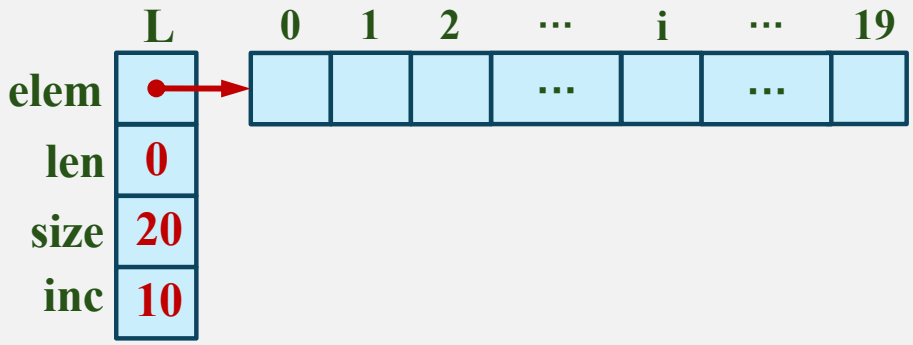
顺序表定义为记录的指针类型SqList和List，两个类型名等价，前者Sq打头以区分采用不同存储表示的线性表类型，后者正是算法2.1和2.2应用的线性表类型名，从中可见，封装做得好的类型，使用者不用了解类型的数据存储结构和操作的实现细节。

结构记录含有4个域，指针elem指示元素存储空间的**基地址**，可作为元素一维数组；len值为线性表当前**长度**。

图2.2(b)不仅展示了顺序表的存储结构，还直观示意了下标和指针对存储结构单元位置的表达。

算法2.3 顺序表的初始化操作

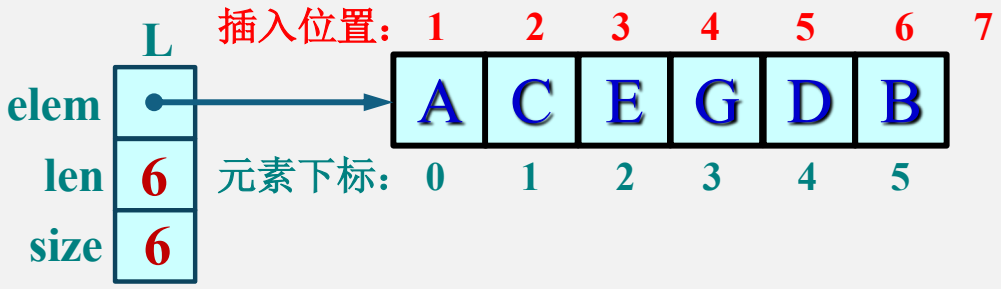
```
                                  20        10
SqList InitList(int size, int inc ) {  // 构造并返回一个空的线性表
  SqList L;
  if (!(L = (SqList)malloc(sizeof(*L)))) exit(OVERFLOW);
  L->elem = (ElemType*)malloc(size*sizeof(ElemType));  // 分配元素空间
  if (!L->elem) exit(OVERFLOW);
  L->len = 0;          // 空表长度为0
  L->size = size;      // 初始分配容量
  L->inc = inc;        // 扩容增量
  return L;        // 返回空表L
}
```



算法2.4　顺序表的插入

```
Status ListInsert(SqList L, int i, ElemType e) {
   if (!L || i<1 || i>L->len+1) return ERROR; // 若插入位置不合法，报错
```

```
} // ListInsert
```



算法分三个步骤：

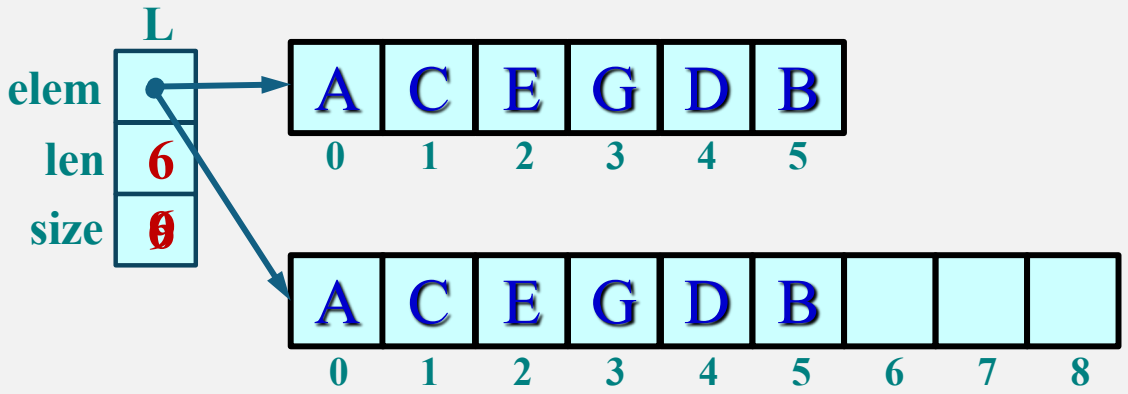
1. 插入位置合法性检查

算法2.4　顺序表的插入(续)

```
Status ListInsert(SqList L, int i, ElemType e) {
    if (!L || i<1 || i>L->len+1) return ERROR; // 若插入位置不合法，报错
    if (L->len >= L->size) {                   // 若当前存储空间已满，则增加空间
        L->elem = (ElemType*)realloc(L->elem,
                    (L->size + L->inc)*sizeof(ElemType));
        if (!L->elem) exit(OVERFLOW);          // 若存储分配失败，报错
        L->size += L->inc;                     // 更新存储容量
    }
    //插入元素 e
      ......
} // ListInsert_Sq
```

2. 若存储空间已满，

则需增加空间

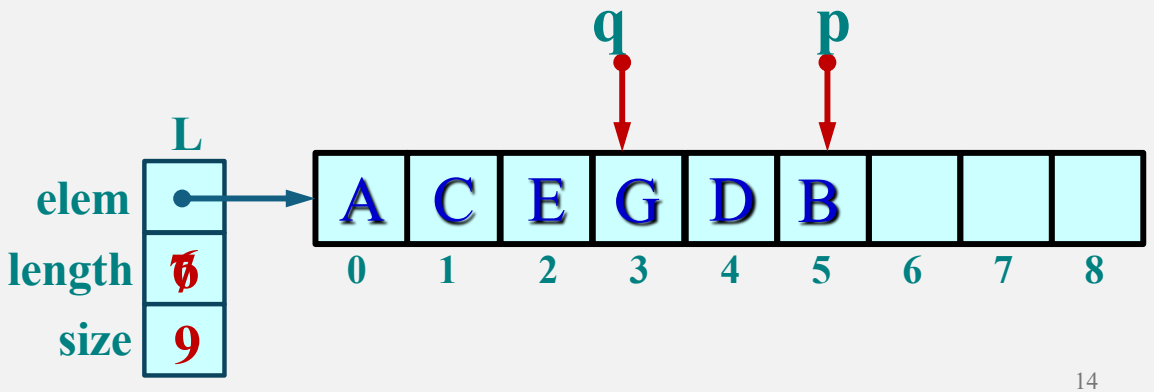


算法2.4　顺序表的插入(续)

```
Status ListInsert(SqList L, int i, ElemType e) {
  ......
  memmove(L->elem+i, L->elem+i-1,
          (L->len-i+1)*sizeof(ElemType));  // “腾位”后移1位
  L->elem[i-1] = e; // 插入 e
  L->len++;         // 表长增 1
  return OK;
} // ListInsert
```

```
int j = i-1;
for (int i=L->len-1; i>=j; --i)
   L->elem[i+1] = L->elem[i];
```

3. 在第 i 位置插入元素 e



算法2.5 删除顺序表的第i元素

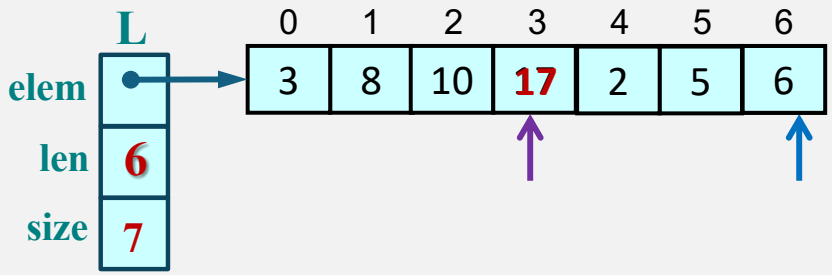
```
ElemType ListDelete(SqList L, int i) {
                                                    4
  if (!L || i<1 || i>L->len) return errV;   // 删除位置i不合法
  ElemType  *p = &(L->elem[i-1]);   // p为被删除元素的位置
  ElemType  e = *p;                 // 被删除元素的值赋给e
  memmove(p, p+1, (L->len-i)*sizeof(ElemType));   // 被删元素之后的所有元素前移
  --L->len;          // 表长减1
  return e;
}
```

（1）首先判断删除位置i是否合法

（2）然后获得被删除元素的位置和值，以及表尾元素的位置；

（3）将被删除元素之后的元素依次左移；

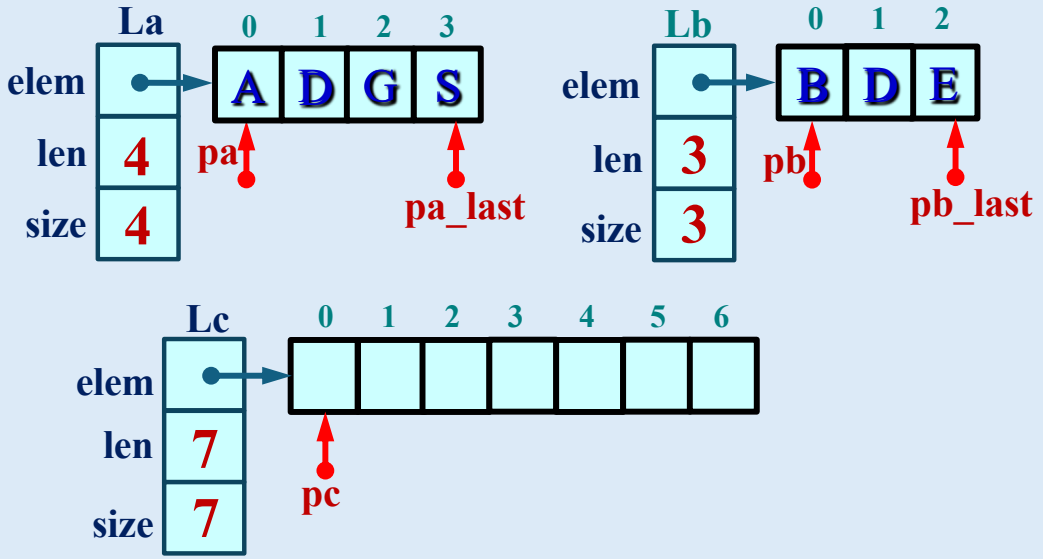
（4）最后将表长减1；



算法2.7 归并有序顺序表

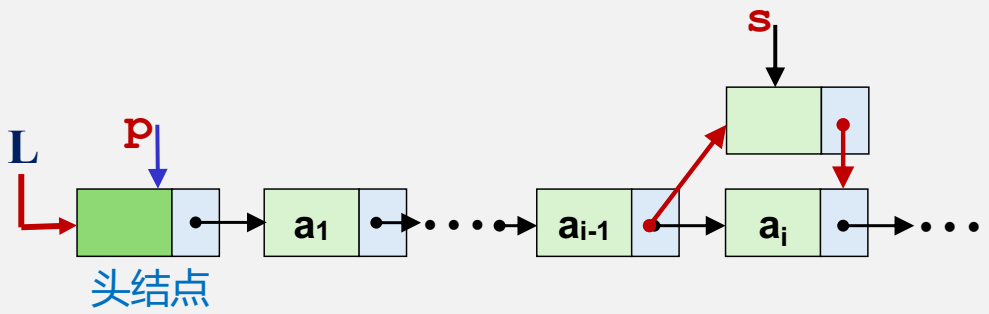
```
SqList MergeList_Sq(SqList La, SqList Lb) { // 归并升序顺序表La和Lb
  SqList Lc;   ElemType *pa, *pb, *pc, *pa_end, *pb_end;
  if (!La || !Lb) return NULL;
  pa = La->elem;  pb = Lb->elem;
  Lc = InitList(La->len + Lb->len, La->inc);
  Lc->len = Lc->size;
  pc = Lc->elem;
  pa_end = La->elem + La->len-1;
  pb_end = Lb->elem + Lb->len-1;
  while (pa<=pa_end && pb<=pb_end)
    if (*pa<=*pb) *pc++ = *pa++;
    else *pc++ = *pb++;
  while (pa<=pa_end) *pc++ = *pa++;
  while (pb<=pb_end) *pc++ = *pb++;
  return Lc;
}
```

1. 初始化Lc
2. 从La和Lb各取一个元素，将较小元素复制到Lc
3. 将La或Lb中未处理完的数据复制到Lc



算法2.9　带头结点的单链表第i个元素前插入元素

```
Status ListInsert_L(LinkList L, int i, ElemType e) {
  if (!L || i<1) return ERROR;   // 若L不存在或i不合法，则报错
  LNode *s, *p = L;    int j = 0;
  while (p && j<i-1) { p = p->next;  ++j; }   // 令p指向第i-1个元素结点
  if (!p) return ERROR;              // 若p空，则i大于表长+1，不合法，报错
  if (!(s = (LinkList)malloc(sizeof(LNode)))) exit(OVERFLOW); // 分配结点
  s->data = e;                                // 新元素值赋给新结点
  s->next = p->next;    p->next = s;    // 新结点插入
  return OK;
}
```



算法2.11 构建单链线性表

构建单链表，需要逐个结点分配存储空间。

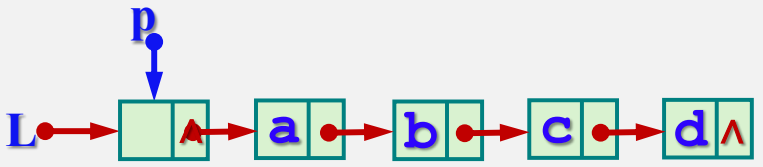
示例：由元素序列

s="a, b, c, d"

构造单链线性表 L。

辅助指针 p 总是指着当前表尾结点，以简化在表尾插入结点。

```
char *s="abcd";
```



初始建头结点

依次在表尾插入各元素结点

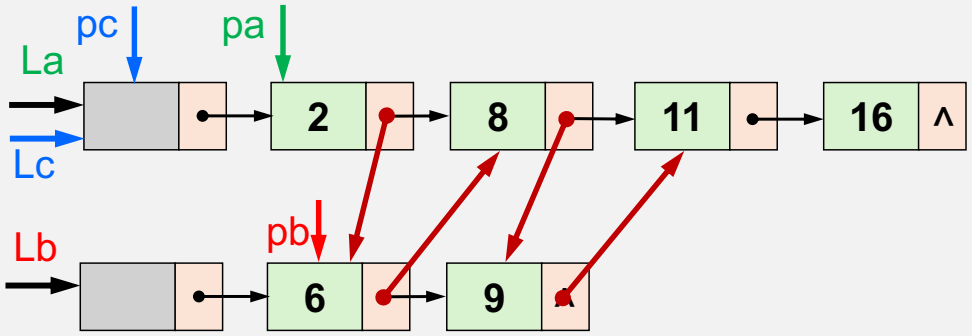
4 "abcd"

```
LinkList CreateList(int n, ElemType s[]) {  // 算法2.11
  // 由元素序列s的头n个元素建立并返回带表头结点的单链线性表
  if (n<0 || !s) return NULL;
  LinkList L, p;
  if (!(L = p = (LinkList)calloc(1, sizeof(LNode)))) exit(OVERFLOW);
  for (int i=0; i<n; ++i) {              // 将 s 的元素逐个插入L的当前表尾
    if (!(p->next = (LinkList)malloc(sizeof(LNode)))) exit(OVERFLOW);
    p = p->next;    p->data = s[i];     // p指向新表尾，存入元素
  }
  p->next = NULL;
}
```

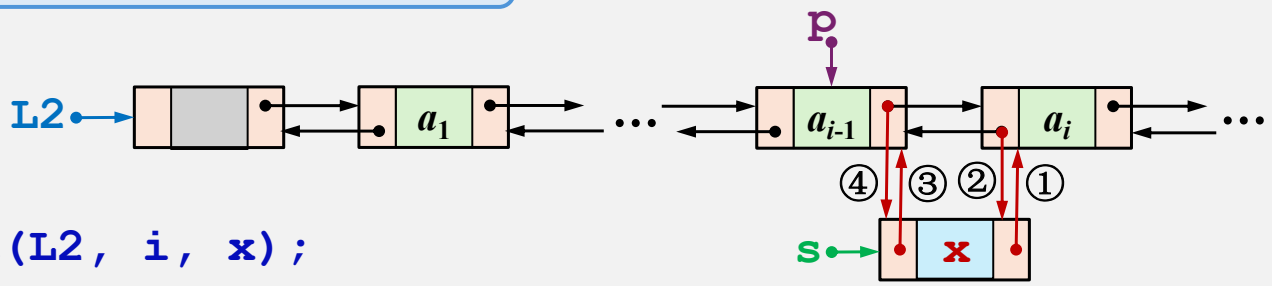
LNode* 可互换

算法2.12　有序单链表的合并

```
LinkList MergeList(LinkList La, LinkList Lb) {
   if (!La || !Lb) return NULL;        // 若La和Lb不存在，则返回NULL，报错
   LinkList *Lc, *pc, *pa = La->next, *pb = Lb->next;
   Lc = pc = La;                       // 用La的头结点作为Lc的头结点
   while(pa && pb)
      if (pa->data <= pb->data) {
         pc->next = pa;   pc = pa;   pa = pa->next;
      } else { pc->next = pb;   pc = pb;   pb = pb->next; }
   pc->next = pa ? pa : pb;
   free(Lb);  return Lc;
}
```



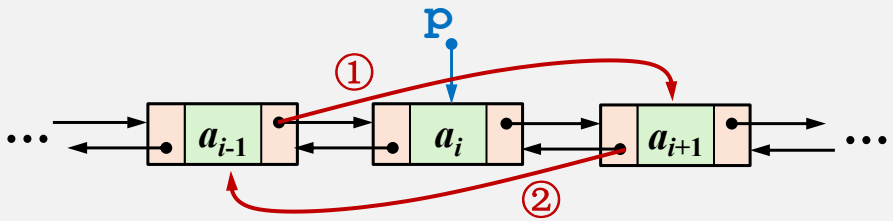
算法2.15　双向链表的插入



ListInsert(L2, i, x);

```
Status ListInsert(DuLinkList L, int i, ElemType e) {  // 算法 2.15
  // 在带头结点双向循环线性表L中第i个元素结点之前插入元素e，1≤i≤表长+1
  if (!L || i<1) return ERROR;  // 若L和i不合法，则报错
  DuLNode *p, *s;
  if (!(p = GetElemPos(L, i-1))) return ERROR; // p指向位序i-1的结点，若P空则报错
  if (!(s = (DuLNode*)malloc(sizeof(DuLNode)))) exit(OVERFLOW);  // 分配插入结点
  s->data = e;              // 置入元素e
  s->next = p->next;        // ①
  p->next->prev = s;        // ②  p结点的后继双向链接为s结点的后继
  s->prev = p;              // ③
  p->next = s;              // ④  s结点双向链接为p结点的后继（和上一行语句不能对调）
  return OK;
}
```

算法2.16　双向链表的删除



x = ListDelete(L2, i);　　e a_i

```
ElemType ListDelete(DuLinkList L, int i) {  // 算法 2.16
  // 删除带头结点的双向循环线性表L的第i个元素，1≤i≤表长
  if (!L || i<1) return errV;                    // 若L和i不合法，则报错
  DuLNode *p;
  if (!(p = GetElemPos(L, i))) return errV;      // 确定第i元素的位置指针p
  ElemType  e = p->data;          // 被删元素值赋给e
  p->prev->next = p->next;        // ① next链删除p结点
  p->next->prev = p->prev;        // ② prev链删除p结点
  free(p);                        // 回收被删的p结点
  return e;                       // 返回被删元素值
}
```

2.4 一元多项式的表示及相加

一般情况下的一元n次多项式可写成

$$P_n(x) = p_1x^{e_1} + p_2x^{e_2} + \cdots + p_mx^{e_m} \qquad (2\text{-}7)$$

其中，p_i是指数为e_i的项的非零系数，且满足

$$0 \leqslant e_1 < e_2 < \cdots < e_m = n$$

若用一个长度为m且每个元素有两个数据项（系数项和指数项）的线性表

$$((p_1, e_1), (p_2, e_2), \ldots, (p_m, e_m)) \qquad (2\text{-}8)$$

便可唯一确定多项式$P_n(x)$。在最坏情况下，$n+1$（$= m$）个系数都不为零，则比只存储每项系数的方案要多存储一倍的数据。但是，对于类似$S(x)$的稀疏多项式，这种表示将大大节省空间。

一元稀疏多项式的顺序表类型定义如下：

```
typedef struct {
  float coef;          // 系数
  int   expn;          // 指数
} Term;                // 项类型
typedef struct {
  Term  *term;         // 存储空间基址
  int   termNum;       // 长度（项数）
} *Poly;   // 一元稀疏多项式（指针）类型（需动态生成，可彻底回收）
```

$S(x)$	0	1	2
coef	1	3	2
expn	0	10000	20000

图2.16 一元稀疏多项式

$S(x) = 1 + 3x^{10000} + 2x^{20000}$

的压缩存储

算法 2.19　构建一元稀疏多项式

构建 $7+3x+9x^8+5x^{17}$ 和 $8x+22x^7-9x^8$

```
Term tsa[] = { {7,0}, {3,1}, {9,8}, {5,17} };
Poly pa = CreatePoly(tsa, 4);

Term tsb[] = { {8,1}, {22,7}, {-9,8}, {5,17} };
Poly pb = CreatePoly(tsb, 3);
```

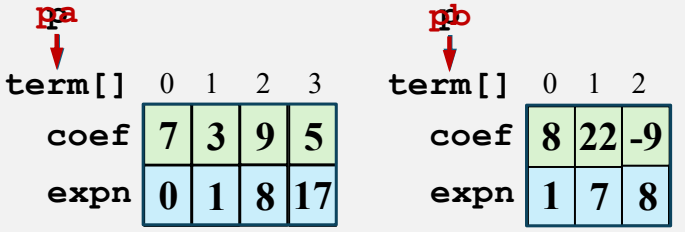


图2.17 一元稀疏多项式构建示例 **(a)**

```
Poly allocPoly(int m) {  // 辅助操作：为一个m项多项式分配存储空间
   Poly Q;
   if (!(Q = (Poly)malloc(sizeof(*Q)))) exit(OVERFLOW);   // 分配结构记录空间
   if (!(Q->term = (Term*)malloc(m*sizeof(Term))))         // 分配m项存储空间
      exit(OVERFLOW); // 若失败，则异常退出
   return Q; // 返回分配了空间的"空"多项式
}

Poly CreatePoly(Term ts[], int m) {  // 由ts前m项构建并返回一元稀疏多项式
   Poly P = allocPoly(m));                              // 分配m项空间
   for (int i=0; i<m; i++) P->term[i] = ts[i];          // 依次存入m项（系数和指数）
   P->termNum = m;                                      // 保存项数
   return P;
}
```

算法 2.21　一元稀疏多项式相加

```
Poly AddPoly(Poly pa, Poly pb) {  // 两个一元稀疏多项式相加，返回和多项式
  int i=0, j=0, k=0;    float c;
  pam = pa->termNum;    pbn = pb->termNum;    // 两个多项式的项数
  Poly pc = allocPoly(pam + pbn);    // 按最大需求预分配和多项式pc存储空间
  while (i < pam && j < pbn) {        // 依次比较两多项式的项的指数
    switch (Compare(pa->term[i].expn, pb->term[j].expn)) { // 比较指数
      case -1: pc->term[k++] = pa->term[i++];  break;    // pa项指数较小
      case  1: pc->term[k++] = pb->term[j++];  break;    // pb项指数较小
      case  0: c = pa->term[i].coef + pb->term[j].coef; // 相等则系数相加
               if (c!=0) {      // 系数和不为0，构建和项，并添加到pc
                 pc->term[k].expn = pa->term[i].expn;
                 pc->term[k].coef = c;
                 k++;
               } // 系数和为0则消去此项
               i++;   j++;       // pa和pb均取下一项
    }//switch
    pc->termNum++;  // 与k同步，支持AnyviewC实时可视跟踪
  }
  if (i == pam)     // pa已处理完，将pb剩余部分添加到pc
    { memcpy(pc->term+k, pb->term+j, (pbn-j)*sizeof(Term));  k += pbn-j; }
  if (j == pbn)     // pb已处理完，将pa剩余部分添加到pc
    { memcpy(pc->term+k, pa->term+i, (pam-i)*sizeof(Term));  k += pam-i; }
  if (k < pam + pbn) // 若pc的实际项数k小于预分配的项数，则按实际调整空间
    if (NULL == (pc->term = (Term*)realloc(pc->term, k*sizeof(Term)))) return NULL;
  pc->termNum = k;  return pc;  // 和多项式的项数，返回所求的和多项式
}
```

